

Servlet & JSP

Steps to Create a First Servlet :-

1. Create a Dynamic Web Project in Eclipse or any IDE.
2. Configure the Apache Tomcat server with your project.
3. Create a new Servlet class inside the src folder.
4. Extend the class from the HttpServlet class.
5. Override the doGet() or doPost() method.
6. Use the response.setContentType("text/html") method to set the output type.
7. Use PrintWriter out = response.getWriter(); to write output.
8. Print HTML output using out.println() statements.
9. Configure the servlet in the web.xml file using <servlet> and <servlet-mapping> tags.
10. Create an HTML file to send a request to the servlet using a form or link.

HTTP Request in Servlet :-

- HTTP (Hypertext Transfer Protocol) is used for communication between client and server.
- Each request uses a specific HTTP method for different operations.
- In servlets, each method is handled by a matching method in HttpServlet class.
- The servlet container automatically calls the correct method based on the client request.

Types of HTTP Request in Servlet :-

1. doGet Method :-

- **Description:** Handles HTTP GET requests.
- **Use Case:** Ideal for retrieving information from the server.

Parameters are appended to the URL.

```
@WebServlet("/example")
public class MyServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // Handling GET requests
        PrintWriter out = response.getWriter();
        out.println("Hello, this is a GET request response!");
    }
}
```

2. doPost Method :-

- **Description:** Handles HTTP POST requests.
- **Use Case:** Used for submitting data to be processed to a specified resource. Commonly used for form submissions.

```
@WebServlet("/example")
public class MyServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // Handling POST requests
        String data = request.getParameter("data"); // Retrieve data from the req
        // Process the data
        PrintWriter out = response.getWriter();
        out.println("Received data: " + data);
    }
}
```

3. doPut Method :-

- **Description:** Handles HTTP PUT requests.
- **Use Case:** Typically used for updating a resource on the server. The entire representation of the resource is sent in the request.

```
@WebServlet("/example")
public class MyServlet extends HttpServlet {
    protected void doPut(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        // Handling PUT requests
        // Extract data from the request and update a resource
        PrintWriter out = response.getWriter();
        out.println("Resource updated successfully!");
    }
}
```

4. doDelete Method:

- **Description:** Handles HTTP DELETE requests.
- **Use Case:** Used to request that a resource be removed. Can delete a specified resource on the server.

```
@WebServlet("/example")
public class MyServlet extends HttpServlet {
    protected void doDelete(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
        // Handling DELETE requests
        // Delete a resource based on the request
        PrintWriter out = response.getWriter();
        out.println("Resource deleted successfully!");
    }
}
```

Difference between doGet() and doPost() in Servlet :-

Basis	doGet()	doPost()
Purpose	Used to handle HTTP GET requests.	Used to handle HTTP POST requests.
Data Transmission	Data is sent through the URL as query parameters.	Data is sent in the request body.
Visibility	Data is visible in the browser's address bar.	Data is not visible in the URL.
Data Size Limit	Limited amount of data can be sent (URL length limit).	Large amounts of data can be sent.
Caching	GET requests can be cached by the browser.	POST requests are not cached.
Bookmarking	Can be bookmarked because parameters are in the URL.	Cannot be bookmarked.
Use Case	Used for retrieving data from the server.	Used for sending or submitting data to the server.
Idempotency	Generally idempotent (no change in server state).	Non-idempotent (may change server state).

Example :-

Addition.java

```
package Demo;

import java.io.IOException;

@WebServlet("/Addition")
public class Addition extends HttpServlet {

    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        int n1=Integer.parseInt(request.getParameter("num1"));
        int n2=Integer.parseInt(request.getParameter("num2"));

        int result=n1+n2;

        System.out.println("Addition= "+result);
    }
}
```

web.xml Configuration (if annotation not used) :-

```
http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd (xsi:schemaLocation with catalog)
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd">

    <servlet>
        <servlet-name>abc</servlet-name>
        <servlet-class>Demo.Addition</servlet-class>
    </servlet>

    <servlet-mapping>
        <servlet-name>abc</servlet-name>
        <url-pattern>/add</url-pattern>
    </servlet-mapping>

</web-app>
```

HTML File (index.html) :-

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>

    <h1>Welcome To Advanced Java</h1>

    <form action="add" method="post">
        <input type="number" placeholder="Enter number 1" name="num1"><br><br>
        <input type="number" placeholder="Enter number 2" name="num2"><br><br>

        <button type="submit">Submit</button>
    </form>

</body>
</html>
```

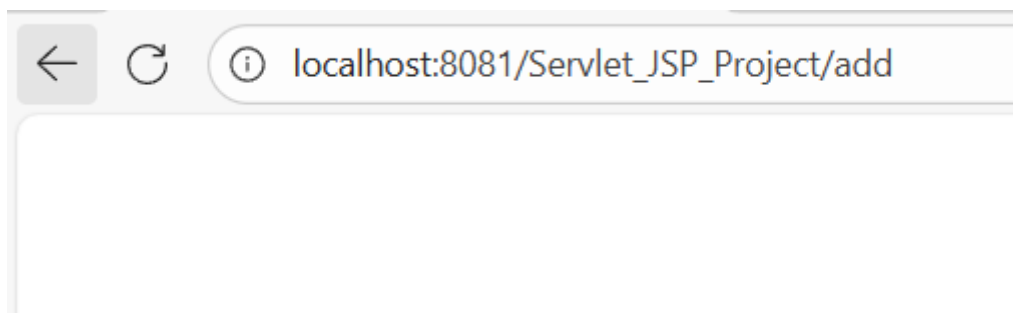
Output :-

HTML Page -



A screenshot of a web browser window. The address bar shows 'localhost:8081/Servlet_JSP_Project/index.html'. The page content includes a heading 'Welcome To Advanced Java' in a large, bold, black serif font. Below the heading are two text input fields. The first field contains the number '5' and the second field contains the number '4'. Below these fields is a button labeled 'Submit'.

Web Page –



A screenshot of a web browser window. The address bar shows 'localhost:8081/Servlet_JSP_Project/add'. The page content is empty.

Console Page -

Addition=9

Steps to Create a JSP Page in a Servlet Project :-

1. Create a Dynamic Web Project in Eclipse or any Java IDE.
2. Configure the Apache Tomcat server with your project.
3. Inside the WebContent folder, create a new JSP file (for example, index.jsp).
4. Write basic HTML structure in the JSP file.
5. Add JSP scripting elements if needed (<% %>, <%= %>, or <%! %>).
6. Create a Servlet class inside the src folder.
7. Extend the servlet class from HttpServlet.
8. In the servlet, process user input or perform backend logic.
9. Use request.setAttribute() to send data from servlet to JSP.
10. Forward the request from servlet to JSP using
RequestDispatcher rd = request.getRequestDispatcher("index.jsp");
11. Call rd.forward(request, response); to display the JSP page.
12. The JSP page receives data from the servlet and displays it in the browser.

Example: -

HTML File (index.html) :-

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>

  <h1>Welcome To Advanced Java</h1>

  <form action="addition.jsp" method="post">
    <input type="number" placeholder="Enter number 1" name="num1"><br><br>
    <input type="number" placeholder="Enter number 2" name="num2"><br><br>

    <button type="submit">Submit</button>
  </form>

</body>
</html>
```

JSP File (addition.jsp) :-

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
    <%
        int n1=Integer.parseInt(request.getParameter("num1"));
        int n2=Integer.parseInt(request.getParameter("num2"));

        int result=n1+n2;

        out.println("Addition= "+result);
    %>
</body>
</html>
```

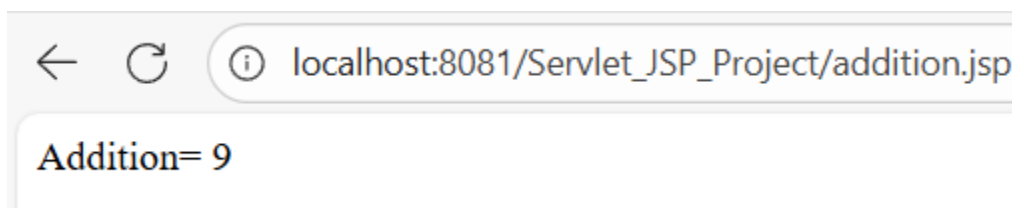
Output :-

HTML Page –



A screenshot of a web browser window. The address bar shows 'localhost:8081/Servlet_JSP_Project/index.html'. The main content area displays 'Welcome To Advanced Java' in a large, bold, black serif font. Below the text, there are two text input fields. The first field contains the number '5' and the second field contains the number '4'. Below these fields is a button labeled 'Submit'.

JSP Page –



A screenshot of a web browser window. The address bar shows 'localhost:8081/Servlet_JSP_Project/addition.jsp'. The main content area displays 'Addition= 9' in a black serif font.

Task – Registration Form using JDBC :-

JSP File (index.jsp) :-

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
    <h1>Registration Form</h1>

    <form action="registration.jsp" method="post">
        <input type="text" name="fullname" placeholder="Enter your fullname"><br><br>
        <input type="email" name="email" placeholder="Enter your email"><br><br>
        <input type="password" name="password" placeholder="Enter your password"><br><br>
        <input type="number" name="contact" placeholder="Enter your contact number"><br><br>

        <button type="submit">Submit</button>
    </form>
</body>
</html>
```

JSP File (registration.jsp) :-

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<%@ page import="java.sql.*" %>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
</head>
<body>
    <%
        String fn=request.getParameter("fullname");
        String em=request.getParameter("email");
        String pss=request.getParameter("password");
        String cn=request.getParameter("contact");

        Class.forName("org.postgresql.Driver");

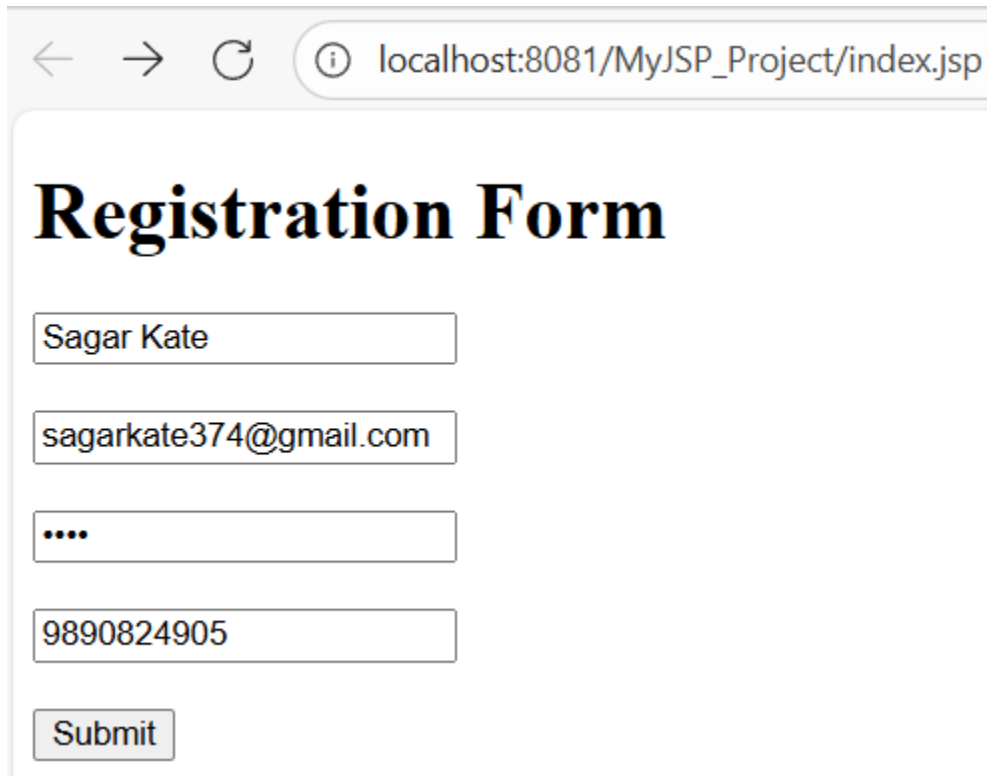
        String URL="jdbc:postgresql://localhost:5433/javabatch";
        String username="postgres";
        String password="4518";

        Connection con=DriverManager.getConnection(URL,username,password);

        String q = "insert into student(fullname,email,password,contact)values(?,?,?,?)";
        PreparedStatement ps = con.prepareStatement(q);
        ps.setString(1, fn);
        ps.setString(2, em);
        ps.setString(3, pss);
        ps.setString(4, cn);
        ps.executeUpdate();
        out.print("Registration Successfully Completed");
    %>
</body>
</html>
```

Output :-

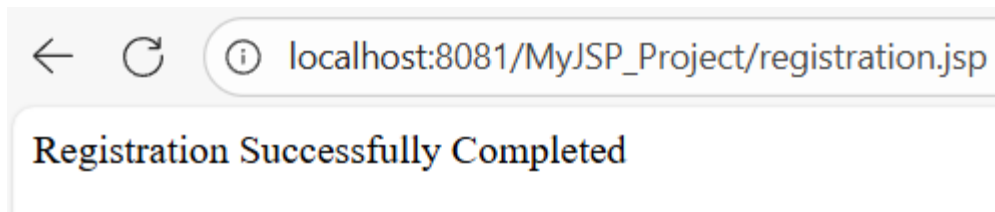
Index.jsp Page –



The screenshot shows a web browser window with the address bar displaying "localhost:8081/MyJSP_Project/index.jsp". The page title is "Index.jsp Page –". The main content is a "Registration Form" with the following fields:

- Fullname: Sagar Kate
- Email: sagarkate374@gmail.com
- Password:
- Contact: 9890824905
- Submit button

Registration.jsp Page –



The screenshot shows a web browser window with the address bar displaying "localhost:8081/MyJSP_Project/registration.jsp". The page title is "Registration.jsp Page –". The main content is a message: "Registration Successfully Completed".

Database – student

	id integer 🔒	fullname character varying (100) 🔒	email character varying (100) 🔒	password character varying (100) 🔒	contact character varying (100) 🔒
1	[null]	Sagar Kate	sagarkate374@gmail.c...	5896	9890824905